Universidade de Brasília

Técnicas de Programação 1 (CIC0197)

Modelo de Arquitetura

Sistema de gestão de projetos pelo método Kanban

Estilo em camadas · 15 módulos · 10 interfaces · diagrama de componentes UML 2

SUMÁRIO

1 Visão geral 2
2 Diagrama de componentes 3
3 Responsabilidades dos módulos 4
4 Interfaces 6
5 Convenções das interfaces 9

EQUIPE

Augusto Queiroz

Flávio de Matos

Pedro Barreto Cavalcante do Amaral

Brasília, 2026

1 Visão geral

O sistema de gestão de projetos pelo método Kanban adota o **estilo arquitetural em camadas**, com duas camadas: a **camada de apresentação** e a **camada de serviço**. Cada camada é decomposta em módulos, e os módulos interagem exclusivamente por meio de serviços especificados em interfaces: classes abstratas C++ declaradas em `headers/interfaces.hpp`, compostas somente por métodos virtuais puros.

- **Camada de apresentação** — interface com o usuário (menus, leitura de dados, exibição de resultados e mensagens) e **validação dos dados de entrada**. A validação acontece ao montar os objetos domínio com `setValor`: quando o formato é inválido, o domínio lança `std::invalid_argument`, tratada nesta camada.
- **Camada de serviço** — **lógica de negócio** (papéis GESTOR e DESENVOLVEDOR, unicidade das chaves, exclusões sem inconsistência, transições de estado dos cartões, limite de WIP e métricas) e **armazenamento dos dados** em contêineres em memória, agrupados no subpacote Armazenamento.

> **Regra de dependência**
>
> - A apresentação depende **somente** das interfaces `IServico*`; nunca das controladoras concretas `CntrServico*` nem do armazenamento.
> - A camada de serviço **não conhece** a apresentação: nenhuma dependência aponta do serviço para a apresentação.
> - O módulo Controle depende apenas das interfaces `IApresentacao*`.
> - O `main` monta as camadas: instancia as controladoras concretas, injeta cada controladora de serviço na apresentação correspondente com `setCntrServico*` e entrega as controladoras de apresentação ao Controle.

A arquitetura de referência do enunciado (APRESENTAÇÃO → SERVIÇO → PESSOAS, PROJETOS, QUADROS, CARTÕES DE ATIVIDADES) foi refinada em módulos, um por entidade, mantendo a mesma direção das dependências:

Tabela 1 – Arquitetura de referência do enunciado e módulos deste modelo

Elemento de referência	Pacote	Módulos
APRESENTAÇÃO	Camada de Apresentação	Controle, Autenticação, Pessoa, Projeto, Quadro, Cartão
SERVIÇO	Camada de Serviço	Autenticação, Pessoa, Projeto, Quadro, Cartão
PESSOAS, PROJETOS, QUADROS, CARTÕES DE ATIVIDADES	Armazenamento	Pessoas, Projetos, Quadros, Cartões de Atividade

> **Exemplo de interação: mover um cartão (serviço 21)**
>
> 1. O Controle chama `IApresentacaoCartao::executar(usuario)`.
> 2. A apresentação de cartões lê o identificador, monta o domínio `Identificador` (validação) e chama `IServicoCartao::mover(usuario, identificador)`.
> 3. O serviço de cartões consulta o quadro (limite de WIP), atualiza estado e timestamps no contêiner de cartões e retorna `true` ou `false`.
> 4. A apresentação exibe o resultado ou a mensagem de recusa.

2 Diagrama de componentes

Componentes UML 2 em pacotes por camada, com colunas alinhadas por entidade: cada interface requerida (socket) encaixa na interface provida (lollipop) correspondente.

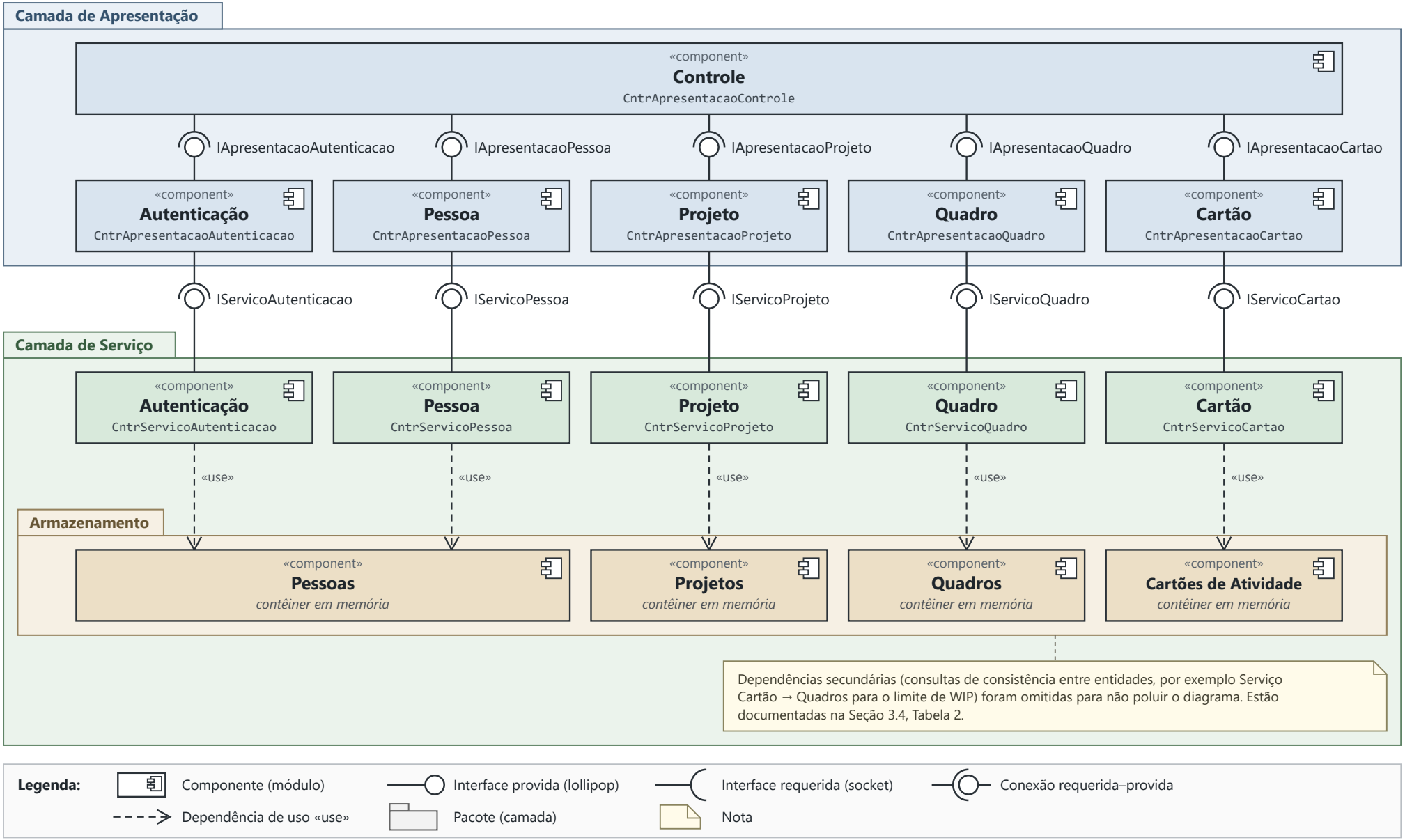


Figura 1 – Diagrama de componentes UML do sistema

3 Responsabilidades dos módulos

Os 15 módulos estão agrupados por camada. Para cada um são indicadas a classe que o implementa, as interfaces que provê e requer e as dependências de armazenamento; em seguida, a responsabilidade do módulo.

3.1 Camada de apresentação

Controle `CntrApresentacaoControle`

Requer `IApresentacaoAutenticacao`, `IApresentacaoPessoa`, `IApresentacaoProjeto`, `IApresentacaoQuadro`, `IApresentacaoCartao`

Ponto de entrada da interação com o usuário. Exibe o menu inicial, que permite criar conta (via `IApresentacaoPessoa::cadastrar`) ou autenticar (via `IApresentacaoAutenticacao::executar`). Após a autenticação, exibe o menu principal e delega cada opção ao módulo correspondente por `executar`, passando o Email do usuário autenticado. Não acessa a camada de serviço.

Autenticação `CntrApresentacaoAutenticacao`

Provê `IApresentacaoAutenticacao` · Requer `IServicoAutenticacao`

Solicita email e senha, monta os domínios `Email` e `Senha` com `setValor` (tratando `std::invalid_argument`) e chama `IServicoAutenticacao::autenticar`. Em caso de sucesso devolve ao Controle o Email do usuário autenticado; caso contrário, informa a falha.

Pessoa `CntrApresentacaoPessoa`

Provê `IApresentacaoPessoa` · Requer `IServicoPessoa`

Oferece a criação de conta sem sessão (`cadastrar`, serviço 1) e o menu de pessoas (serviços 1 a 5). Lê e valida email, nome, senha e papel, monta a entidade `Pessoa`, chama `IServicoPessoa` e exibe os dados, a lista de emails ou a mensagem de recusa.

Projeto `CntrApresentacaoProjeto`

Provê `IApresentacaoProjeto` · Requer `IServicoProjeto`

Menu de projetos (serviços 6 a 10). Lê e valida identificador, nome, descrição, início e término, chama `IServicoProjeto` e apresenta o resultado; quando o serviço recusa (por exemplo, usuário que não é GESTOR ou projeto com quadros), exibe mensagem ao usuário.

Quadro `CntrApresentacaoQuadro`

Provê `IApresentacaoQuadro` · Requer `IServicoQuadro`

Menu de quadros (serviços 11 a 15). Lê e valida o projeto ao qual o quadro pertence, identificador, nome e limite de WIP, chama `IServicoQuadro` e lista os quadros de um projeto.

Cartão `CntrApresentacaoCartao`

Provê `IApresentacaoCartao` · Requer `IServicoCartao`

Menu de cartões de atividade (serviços 16 a 26): criar, ler, atualizar, excluir, listar e mover cartões, e consultar as métricas (tempo de ciclo, lead time e respectivas médias, em minutos, e WIP do quadro). Valida os dados de entrada e formata os resultados de `IServicoCartao`.

3.2 Camada de serviço

Autenticação `CntrServicoAutenticacao`

Provê `IServicoAutenticacao` · **Usa** Pessoas

Verifica se existe pessoa com o email e a senha informados, consultando o contêiner de pessoas. Não mantém sessão: o Email do usuário autenticado fica com a camada de apresentação e é repassado nas chamadas seguintes.

Pessoa `CntrServicoPessoa`

Provê `IServicoPessoa` · **Usa** Pessoas · **Consulta** Projetos, Cartões de Atividade

Mantém as pessoas (serviços 1 a 5) e garante a unicidade do Email. `criar` funciona sem sessão; `atualizar` não altera a chave. Recusa excluir quem é gestor de projeto ou desenvolvedor de cartão e recusa a troca de papel que deixaria um projeto sem GESTOR ou um cartão sem DESENVOLVEDOR.

Projeto `CntrServicoProjeto`

Provê `IServicoProjeto` · **Usa** Projetos · **Consulta** Pessoas, Quadros

Mantém os projetos (serviços 6 a 10). Somente GESTOR cria, atualiza e exclui; o gestor do projeto é o usuário autenticado que o criou. Garante a unicidade do Identificador e recusa excluir projeto que possui quadros (sem cascata).

Quadro `CntrServicoQuadro`

Provê `IServicoQuadro` · **Usa** Quadros · **Consulta** Pessoas, Projetos, Cartões de Atividade

Mantém os quadros (serviços 11 a 15), sempre vinculados a um projeto existente. Somente GESTOR cria, atualiza e exclui. Recusa novo limite menor que o WIP atual e recusa excluir quadro que possui cartões; lista os quadros de um projeto.

Cartão `CntrServicoCartao`

Provê `IServicoCartao` · **Usa** Cartões de Atividade · **Consulta** Pessoas, Quadros

Mantém os cartões (serviços 16 a 26). Cria o cartão em A FAZER com entrada = momento atual, atribuído a uma pessoa com papel DESENVOLVEDOR. `mover` faz A FAZER → FAZENDO (registra início) e FAZENDO → FEITO (registra término), respeitando o limite de WIP do quadro. Calcula tempo de ciclo (término − início), lead time (término − entrada), as médias por quadro e o WIP.

3.3 Armazenamento (dentro da camada de serviço)

Cada contêiner guarda as entidades em memória, indexadas pela chave primária, e é acessado somente pela camada de serviço: a apresentação nunca o acessa. A forma de compartilhamento dos contêineres entre as controladoras de serviço será definida na implementação.

Pessoas *contêiner em memória*

Usado por Serviço Autenticação, Serviço Pessoa · **Consultado por** Serviço Projeto, Serviço Quadro, Serviço Cartão

Contêiner em memória das pessoas cadastradas, indexado pelo Email (chave primária).

Projetos *contêiner em memória*

Usado por Serviço Projeto · **Consultado por** Serviço Pessoa, Serviço Quadro

Contêiner em memória dos projetos, indexado pelo Identificador (chave primária).

Quadros *contêiner em memória*

Usado por Serviço Quadro · **Consultado por** Serviço Projeto, Serviço Cartão

Contêiner em memória dos quadros, indexado pelo Identificador (chave primária).

Cartões de Atividade *contêiner em memória*

Usado por Serviço Cartão · **Consultado por** Serviço Pessoa, Serviço Quadro

Contêiner em memória dos cartões de atividade, indexado pelo Identificador (chave primária).

3.4 Dependências secundárias (consultas de consistência)

O diagrama mostra apenas a dependência principal de cada módulo de serviço (o contêiner da própria entidade). Para aplicar as regras de negócio, os módulos também consultam contêineres de outras entidades:

Tabela 2 – Dependências secundárias da camada de serviço para o armazenamento

Módulo de serviço	Consulta	Motivo
Serviço Pessoa	Projetos	Recusar exclusão ou troca de papel de quem é gestor de algum projeto.
	Cartões de Atividade	Recusar exclusão ou troca de papel de quem é desenvolvedor de algum cartão.
Serviço Projeto	Pessoas	Conferir se o usuário tem papel GESTOR (criar, atualizar, excluir).
	Quadros	Recusar exclusão de projeto que possui quadros.
Serviço Quadro	Pessoas	Conferir o papel do usuário (somente GESTOR altera).
	Projetos	Conferir se o projeto do quadro existe.
	Cartões de Atividade	Recusar exclusão de quadro com cartões; exigir novo limite ≥ WIP atual.
Serviço Cartão	Pessoas	Conferir papel GESTOR do usuário; conferir se o desenvolvedor existe e tem papel DESENVOLVEDOR.
	Quadros	Conferir se o quadro existe; respeitar o limite de WIP ao mover para FAZENDO.

4 Interfaces

As 10 interfaces estão declaradas como classes abstratas em `headers/interfaces.hpp`. A coluna Serviços traz a numeração do enunciado (1 a 26); criar conta corresponde ao serviço 1 e a autenticação não é numerada. As interfaces `IApresentacao*` atendem os mesmos serviços da `IServico*` que o módulo requer.

Tabela 3 – Interfaces entre os módulos

Interface	Provida por	Requerida por	Serviços	Métodos
`IServicoAutenticacao`	Serviço Autenticação	Apres. Autenticação	autenticação (sem nº)	autenticar
`IServicoPessoa`	Serviço Pessoa	Apres. Pessoa	1–5	criar (1), ler (2), atualizar (3), excluir (4), listar (5)
`IServicoProjeto`	Serviço Projeto	Apres. Projeto	6–10	criar (6), ler (7), atualizar (8), excluir (9), listar (10)
`IServicoQuadro`	Serviço Quadro	Apres. Quadro	11–15	criar (11), ler (12), atualizar (13), excluir (14), listar (15)

Interface	Provida por	Requerida por	Serviços	Métodos
IServicoCartao	Serviço Cartão	Apres. Cartão	16–26	criar (16), ler (17), atualizar (18), excluir (19), listar (20), mover (21), lerTempoCiclo (22), lerTempoCicloMedio (23), lerLeadTime (24), lerLeadTimeMedio (25), lerWIP (26)
IApresentacaoAutenticacao	Apres. Autenticação	Controle	autenticação (sem nº)	executar(Email&), setCntrServicoAutenticacao
IApresentacaoPessoa	Apres. Pessoa	Controle	1–5 (cadastrar = 1, sem sessão)	executar(const Email&), cadastrar(), setCntrServicoPessoa
IApresentacaoProjeto	Apres. Projeto	Controle	6–10	executar(const Email&), setCntrServicoProjeto
IApresentacaoQuadro	Apres. Quadro	Controle	11–15	executar(const Email&), setCntrServicoQuadro
IApresentacaoCartao	Apres. Cartão	Controle	16–26	executar(const Email&), setCntrServicoCartao

4.1 Assinaturas C++

Transcritas de `headers/interfaces.hpp` (comentários Doxygen omitidos). O comentário ao final de cada método indica o número do serviço.

```
class IServicoAutenticacao { // serviços: autenticação (sem nº)
public:
    virtual bool autenticar(const Email& email, const Senha& senha) = 0;
    virtual ~IServicoAutenticacao() = 0;
};
```

```
class IServicoPessoa { // serviços: 1–5
public:
    virtual bool criar(const Pessoa& pessoa) = 0; // 1
    virtual bool ler(const Email& usuario, const Email& email, Pessoa& pessoa) = 0; // 2
    virtual bool atualizar(const Email& usuario, const Pessoa& pessoa) = 0; // 3
    virtual bool excluir(const Email& usuario, const Email& email) = 0; // 4
    virtual bool listar(const Email& usuario, std::vector<Email>& lista) = 0; // 5
    virtual ~IServicoPessoa() = 0;
};
```

```
class IServicoProjeto { // serviços: 6–10
public:
    virtual bool criar(const Email& usuario, const Projeto& projeto) = 0; // 6
    virtual bool ler(const Email& usuario, const Identificador& identificador,
                     Projeto& projeto) = 0; // 7
    virtual bool atualizar(const Email& usuario, const Projeto& projeto) = 0; // 8
    virtual bool excluir(const Email& usuario, const Identificador& identificador) = 0; // 9
    virtual bool listar(const Email& usuario, std::vector<Identificador>& lista) = 0; // 10
    virtual ~IServicoProjeto() = 0;
};
```

```
class IServicoQuadro { // serviços: 11–15
public:
    virtual bool criar(const Email& usuario, const Identificador& projeto,
                       const Quadro& quadro) = 0; // 11
    virtual bool ler(const Email& usuario, const Identificador& identificador,
                     Quadro& quadro) = 0; // 12
    virtual bool atualizar(const Email& usuario, const Quadro& quadro) = 0; // 13
    virtual bool excluir(const Email& usuario, const Identificador& identificador) = 0; // 14
    virtual bool listar(const Email& usuario, const Identificador& projeto,
                        std::vector<Identificador>& lista) = 0; // 15
    virtual ~IServicoQuadro() = 0;
};
```

```
class IServicoCartao { // serviços: 16–26
public:
    virtual bool criar(const Email& usuario, const Identificador& quadro,
                       const Email& desenvolvedor, const CartaoAtividade& cartao) = 0; // 16
    virtual bool ler(const Email& usuario, const Identificador& identificador,
                     CartaoAtividade& cartao) = 0; // 17
    virtual bool atualizar(const Email& usuario, const CartaoAtividade& cartao) = 0; // 18
    virtual bool excluir(const Email& usuario, const Identificador& identificador) = 0; // 19
    virtual bool listar(const Email& usuario, const Identificador& quadro,
                        std::vector<Identificador>& lista) = 0; // 20
    virtual bool mover(const Email& usuario, const Identificador& identificador) = 0; // 21
    virtual bool lerTempoCiclo(const Email& usuario, const Identificador& identificador,
                               double& minutos) = 0; // 22
    virtual bool lerTempoCicloMedio(const Email& usuario, const Identificador& quadro,
                                    double& minutos) = 0; // 23
    virtual bool lerLeadTime(const Email& usuario, const Identificador& identificador,
                             double& minutos) = 0; // 24
    virtual bool lerLeadTimeMedio(const Email& usuario, const Identificador& quadro,
                                  double& minutos) = 0; // 25
    virtual bool lerWIP(const Email& usuario, const Identificador& quadro, int& wip) = 0; // 26
    virtual ~IServicoCartao() = 0;
};
```

```
class IApresentacaoAutenticacao { // serviços: autenticação (sem nº)
public:
    virtual bool executar(Email& email) = 0;
    virtual void setCntrServicoAutenticacao(IServicoAutenticacao* cntr) = 0;
    virtual ~IApresentacaoAutenticacao() = 0;
};
```

```
class IApresentacaoPessoa { // serviços: 1–5 (cadastrar = 1, sem sessão)
public:
    virtual void executar(const Email& usuario) = 0;
    virtual bool cadastrar() = 0;
    virtual void setCntrServicoPessoa(IServicoPessoa* cntr) = 0;
    virtual ~IApresentacaoPessoa() = 0;
};
```

```
class IApresentacaoProjeto { // serviços: 6–10
public:
    virtual void executar(const Email& usuario) = 0;
    virtual void setCntrServicoProjeto(IServicoProjeto* cntr) = 0;
    virtual ~IApresentacaoProjeto() = 0;
};
```

```
class IApresentacaoQuadro { // serviços: 11–15
public:
    virtual void executar(const Email& usuario) = 0;
    virtual void setCntrServicoQuadro(IServicoQuadro* cntr) = 0;
    virtual ~IApresentacaoQuadro() = 0;
};
```

```
class IApresentacaoCartao { // serviços: 16–26
public:
    virtual void executar(const Email& usuario) = 0;
    virtual void setCntrServicoCartao(IServicoCartao* cntr) = 0;
    virtual ~IApresentacaoCartao() = 0;
};
```

5 Convenções das interfaces

Valem para todas as interfaces de serviço e estão registradas no topo de `headers/interfaces.hpp`:

1. **Retorno bool.** Todo método de serviço retorna `bool`: `true` = operação realizada; `false` = operação recusada (registro inexistente ou duplicado, permissão negada, regra de negócio violada ou exclusão que causaria inconsistência). Se retornar `false`, nada é alterado. O serviço **não lança exceções**; `std::invalid_argument` só aparece na apresentação, ao montar domínios com `setValor`.
2. **Usuário autenticado.** Todo método que exige sessão recebe, como **primeiro parâmetro**, o `Email` do usuário autenticado (`usuario`), usado para consultar o papel (GESTOR ou DESENVOLVEDOR) e para definir o gestor de um projeto. Exceções: `IServicoAutenticacao::autenticar` e `IServicoPessoa::criar`, que funcionam sem sessão (criação de conta).
3. **Entrada.** Parâmetros de entrada são referências constantes (`const &`). Associações são informadas pela chave primária da outra entidade (ex.: criar quadro recebe o `Identificador` do projeto).
4. **Saída.** Parâmetros de saída são referências **não constantes** (`&`), sempre os últimos da lista, e só são preenchidos quando o método retorna `true`.
5. **Listagens.** Preenchem um `std::vector` com as chaves primárias (`Email` ou `Identificador`). Retorno `true` com vetor vazio significa que não há itens.
6. **Chave primária.** `atualizar` localiza o registro pela chave primária contida na entidade recebida e altera somente os demais atributos; a chave nunca é alterada.
7. **Métricas de tempo.** Tempo de ciclo, lead time e médias são `double`, em **minutos**. Retornam `false` se o cartão não está em FEITO ou se o quadro não possui cartão em FEITO.
8. **Exclusão sem cascata.** Exclusões que gerariam inconsistência são recusadas, nunca propagadas: projeto com quadros, quadro com cartões e pessoa vinculada a projeto ou cartão. A ordem de exclusão é cartões → quadros → projeto → pessoa.
9. **Destrutor virtual puro.** Cada interface declara destrutor virtual **puro**, definido vazio (`inline`) no final de `interfaces.hpp`. Assim a interface contém somente métodos virtuais puros e o `delete` por ponteiro de interface é seguro.